Flash Read and Write Experiment

1. NVS Overview

NVS (Non-Volatile Storage) is a **key-value** storage library provided by ESP-IDF, based on a dedicated partition in SPI Flash. NVS encapsulates wear-leveling and power-failure protection on Flash, allowing developers to read and write persistent data like using a "dictionary" without worrying about low-level erase details.

Typical use cases:

Scenario	Description
Wi-Fi Credentials	SSID + password persistence
Device Configuration	Calibration values, thresholds, mode selection
Device Run Counters	Power-on count, runtime duration
User Data Logging	Log entries, sensor history (**this example**)
OTA Status Flags	Firmware version, upgrade flags

This project demonstrates complete NVS CRUD operations via **UART0 interactive menu**: append write, read all, delete by index (auto re-index), clear all.

---

2. Core Concepts

2.1 NVS Key-Value Model

NVS organizes data in a three-level structure: **Namespace** → **Key** → **Value**:

```
Flash Partition
  └─ Namespace "data_store"
       ├─ "count"   → 3   (int32_t)
       ├─ "dat1"    → "Hello"   (string)
       ├─ "dat2"    → "World"   (string)
       └─ "dat3"    → "ESP32"   (string)
```

> Analogy: Namespace ≈ database table, Key ≈ column name, Value ≈ cell.

2.2 NVS Supported Data Types

Type	Functions	Size
Integer	`nvs_set_i8()` / `nvs_set_i16()` / `nvs_set_i32()` / `nvs_set_i64()`	1~8 bytes
String	`nvs_set_str()`	Variable, ≤ 1984 bytes
Binary Blob	`nvs_set_blob()`	Variable, ≤ 1984 bytes

2.3 NVS Operation Flow

```
1. nvs_flash_init()          Initialize NVS partition
2. nvs_open("namespace")     Open namespace, get handle
3. nvs_set_xxx() / nvs_get_xxx()  Write / Read
4. nvs_commit()              Commit write (persist to flash)
5. nvs_close()               Close handle
```

> **Important**: Data is not immediately written to Flash after `nvs_set_xxx()`. You must call `nvs_commit()` to persist. Power loss immediately after write may cause data loss.

2.4 NVS Internal Principle

NVS uses a **log-structured** storage on Flash — new data is appended, old data is marked invalid, and background GC (garbage collection) runs automatically:

```
Flash Sector 1:  [dat1=Hello][dat2=World][count=2][..........]
Flash Sector 2:  [dat2=World_v2]                       ← Update: append new record
Flash Sector 3:  [.....................................]  ← Free / GC
```

> This design naturally supports power-failure protection: writing new records does not erase old data, and interrupted operations can rollback to the previous version on restart.

---

3. ESP32-S3 NVS Hardware Specifications

Feature	Parameter
Storage Medium	SPI Flash (internal or external)
Partition Type	`data, nvs`
Minimum Erase Unit	4 KB (1 sector)
Max Namespace Count	Limited by Flash partition size
Max Key Capacity	String/Blob ≤ 1984 bytes
Erase/Write Endurance	≥ 100,000 cycles (Flash physical characteristic)
Power-Failure Protection	Log-structured write naturally supports
Encryption	Optional NVS Encryption

---

4. Project Architecture and Flow

```
app_main()
  ├─ uart_param_config()          Configure UART0: 115200-8N1
  ├─ uart_driver_install()        Install UART driver
  ├─ nvs_init()                   Initialize NVS partition (erase if needed)
  ├─ nvs_get_i32("count")         Read current entry count on boot
  ├─ printf()                     Print interactive menu
  └─ while(1)
       ├─ uart_read_line()        Read one line of user input (echo + backspace)
       └─ switch(cmd)
            ├─ '1' → Append Write     datN = user input, count++
            ├─ '2' → Read All         Iterate dat1..datN and print
            ├─ '3' → Delete by idx     Delete specified index, shift entries forward, count-
-
            └─ '4' → Clear All        nvs_flash_erase() entire partition
```

NVS Key Naming Scheme

Key	Type	Description
`count`	`int32_t`	Current entry count
`dat1`, `dat2`, `dat3`, ...	`string`	User data entries

---

5. Hardware Dependencies

Hardware Connection: None. Interacts with PC serial terminal via UART0 (USB serial).

Required Components: `nvs_flash`, `driver/uart`, `FreeRTOS`

---

6. Source Code Details

6.1 nvs_init() — NVS Partition Initialization

This function handles NVS Flash partition initialization. If `nvs_flash_init()` returns "no free pages" or "version mismatch" errors (common after first flash or partition table change), it automatically erases the entire NVS partition and re-initializes to ensure normal read/write operations.

```
/**
 * @brief  NVS initialization
 */
void nvs_init() {
    // Try to initialize NVS partition
    esp_err_t err = nvs_flash_init();
    // Check for no free pages or new version
    if (err == ESP_ERR_NVS_NO_FREE_PAGES || err == ESP_ERR_NVS_NEW_VERSION_FOUND) {
        nvs_flash_erase();  // Erase NVS space
        err = nvs_flash_init();
    }
}
```

6.2 uart_read_line() — Line Input (with Echo + Backspace)

This function implements a simple line editor: reads user input byte-by-byte from UART0 and echoes to terminal, ends on newline `\n` or `\r`, handles backspace (`\b` or DEL=127) by deleting previous character and echoing erase sequence `\b \b` for interactive terminal input.

```
/**
 * @brief  Read a line from UART (with echo)
 * @param  buf      Receive buffer
 * @param  max_len  Max buffer length
 * @return Number of bytes read (excluding \0)
 */
static int uart_read_line(uint8_t *buf, int max_len) {
```

```
    int total = 0;
    while (total < max_len - 1) {
        uint8_t ch;
        int n = uart_read_bytes(UART_NUM_0, &ch, 1, portMAX_DELAY);
        if (n <= 0) continue;

        if (ch == '\n') {           // LF: end of line
            uart_write_bytes(UART_NUM_0, "\r\n", 2);  // Echo newline
            break;
        }
        if (ch == '\r') {           // CR
            uart_write_bytes(UART_NUM_0, "\r\n", 2);
            // Consume the following \n
            uart_read_bytes(UART_NUM_0, &ch, 1, 1);
            break;
        }
        if (ch == '\b' || ch == 127) {   // Backspace key
            if (total > 0) {
                total--;
                // Erase one char
                uart_write_bytes(UART_NUM_0, "\b \b", 3);
            }
            continue;
        }
        buf[total++] = ch;
        uart_write_bytes(UART_NUM_0, &ch, 1);  // Echo
    }
    buf[total] = '\0';  // Null terminator
    return total;
}
```

6.3 Append Write (Command '1')

After user inputs command `1` and enters text content, the program increments `dataCount` and stores it in NVS with key `datN` (N is current entry number), while updating the `count` key to record total entry count. After writing, `nvs_commit()` is called to persist to Flash.

```
// 1: Append write data
if (cmd == '1') {
    printf("Enter content to save:\n");
    len = uart_read_line(data, BUF_SIZE);
    if (len <= 0) {
        printf("Empty input, skipped\n");
        continue;
    }

    // Open namespace (read-write mode)
    nvs_open("data_store", NVS_READWRITE, &handle);
    dataCount++;
    char key[16];
    sprintf(key, "dat%ld", (long)dataCount);  // Build key name
```

```
    // Write string value
    nvs_set_str(handle, key, (char *)data);
    // Update entry count
    nvs_set_i32(handle, "count", dataCount);
    nvs_commit(handle);   // Commit to flash
    nvs_close(handle);    // Close handle

    printf("Entry %ld saved: %s\n", (long)dataCount, (char *)data);
}
```

6.4 Read All (Command '2')

Opens namespace in read-only mode, reads `count` to get total entry count, then iterates `dat1` ~ `datN` to read and print each entry. If `count=0`, shows "no data".

```
// 2: Read all saved data
else if (cmd == '2') {
    // Open namespace (read-only mode)
    nvs_open("data_store", NVS_READONLY, &handle);
    // Read total entry count
    nvs_get_i32(handle, "count", &dataCount);

    if (dataCount == 0) {
        printf("No data\n");
    } else {
        printf("=== All data (%ld entries) ===\n", (long)dataCount);
        for (int i = 1; i <= dataCount; i++) {
            // Build key name
            char key[16]; sprintf(key, "dat%d", i);
            char buf[256]; size_t buf_len = sizeof(buf);
            // Read string value
            if (nvs_get_str(handle, key, buf, &buf_len) == ESP_OK) {
                printf("%d: %s\n", i, buf);
            }
        }
    }
    nvs_close(handle);
}
```

6.5 Delete by Index (Command '3') — Auto Re-index

After user inputs the entry number to delete, the program validates the range, then shifts all subsequent entries one position forward starting from the delete position, finally erases the leftover last key and updates `count`. Re-indexing ensures `dat1` ~ `datN` remain consecutive with no gaps or disorder.

```
// 3: Delete specified index, then re-index
else if (cmd == '3') {
    printf("Enter entry number to delete:\n");
    len = uart_read_line(data, BUF_SIZE);
    if (len <= 0) continue;
    int delNum = atoi((char *)data);  // Parse index
```

```
    nvs_open("data_store", NVS_READWRITE, &handle);
    nvs_get_i32(handle, "count", &dataCount);

    // Validate index range
    if (delNum < 1 || delNum > dataCount) {
        printf("Invalid number (1-%ld)\n", (long)dataCount);
        nvs_close(handle);
        continue;
    }

    // Shift subsequent entries forward
    for (int i = delNum; i < dataCount; i++) {
        // Current key
        char key_cur[16];  sprintf(key_cur, "dat%d", i);
        // Next key
        char key_next[16]; sprintf(key_next, "dat%d", i + 1);
        char buf[256]; size_t buf_len = sizeof(buf);

        if (nvs_get_str(handle, key_next, buf, &buf_len) == ESP_OK) {
            // Move to previous key
            nvs_set_str(handle, key_cur, buf);
            nvs_erase_key(handle, key_next);      // Erase old key
        }
    }
    // Erase the now-unused last key
    char last_key[16]; sprintf(last_key, "dat%ld", (long)dataCount);
    nvs_erase_key(handle, last_key);

    dataCount--;
    nvs_set_i32(handle, "count", dataCount);  // Update count
    nvs_commit(handle);   // Commit to flash
    nvs_close(handle);    // Close handle

    printf("Entry %d deleted (total %ld)\n", delNum, (long)dataCount);
}
```

6.6 Clear All (Command '4')

Calls `nvs_flash_erase()` to erase the entire NVS partition, then re-initializes and resets `dataCount` to 0. All stored data will be completely cleared.

```
// 4: Clear all NVS storage space
else if (cmd == '4') {
    nvs_flash_erase();   // Erase entire NVS partition
    nvs_init();          // Re-initialize
    dataCount = 0;       // Reset counter
    printf("All data cleared!\n");
}
```

6.7 app_main() — Entry Point

Main function sequentially: UART0 initialization (115200-8N1) → NVS initialization → Read stored entry count on boot → Print interactive menu → Enter `while(1)` loop to wait for user input, dispatch to corresponding CRUD module based on first character `1/2/3/4`.

```
/**
 * @brief  Application entry point
 */
void app_main(void) {
    // UART config 115200 8N1
    uart_config_t uart_config = {
        .baud_rate = 115200,
        .data_bits = UART_DATA_8_BITS,
        .parity = UART_PARITY_DISABLE,
        .stop_bits = UART_STOP_BITS_1,
        .flow_ctrl = UART_HW_FLOWCTRL_DISABLE,
    };
    uart_param_config(UART_NUM_0, &uart_config);
    // Install UART driver
    uart_driver_install(UART_NUM_0, BUF_SIZE * 2, 0, 0, NULL, 0);

    nvs_init();  // Initialize NVS

    nvs_handle_t handle;
    int32_t dataCount = 0;
    // Read stored count on boot
    nvs_open("data_store", NVS_READWRITE, &handle);
    nvs_get_i32(handle, "count", &dataCount);
    nvs_close(handle);

    // Print interactive menu
    printf("\n================================\n");
    printf("   ESP32-S3 NVS Flash Storage\n");
    printf(" 1 = Append Write\n");
    printf(" 2 = Read All Data\n");
    printf(" 3 = Delete Specified Data\n");
    printf(" 4 = Clear All Data\n");
    printf("================================\n");

    while (1) {
        printf("\nCMD> ");
        fflush(stdout);  // Flush output immediately

        len = uart_read_line(data, BUF_SIZE);
        if (len <= 0) { vTaskDelay(10); continue; }
        char cmd = (char)data[0];  // First char as command

        // Dispatch by command
        if (cmd == '1') { /* ... Append write ... */ }
        else if (cmd == '2') { /* ... Read All ... */ }
        else if (cmd == '3') { /* ... Delete by index ... */ }
```

```
        else if (cmd == '4') { /* ... Clear All ... */ }

        vTaskDelay(20);  // Yield CPU
    }
}
```

---

7. Operation Flow

7.1 Build and Flash Process

For detailed flash process: 1. Before Development → 3. Build and Flash

7.2 Normal Interaction Example

```
================================
   ESP32-S3 NVS Flash Storage
 1 = Append Write
 2 = Read All Data
 3 = Delete Specified Data
 4 = Clear All Data
================================

CMD> 1
Enter content to save:
Hello World!
Entry 1 saved: Hello World!

CMD> 1
Enter content to save:
ESP32-S3 NVS Demo
Entry 2 saved: ESP32-S3 NVS Demo

CMD> 2
=== All data (2 entries) ===
1: Hello World!
2: ESP32-S3 NVS Demo

CMD> 3
Enter entry number to delete:
1
Entry 1 deleted (total 1)

CMD> 2
=== All data (1 entries) ===
1: ESP32-S3 NVS Demo

CMD> 4
All data cleared!

CMD> 2
No data
```

```
================================
   ESP32-S3 NVS Flash Storage
 1 = Append Write
 2 = Read All Data
 3 = Delete Specified Data
 4 = Clear All Data
================================

CMD> 1
Enter content to save:
Hello World!
Entry 1 saved: Hello World!

CMD> 1
Enter content to save:
ESP32-S3 NVS Demo
Entry 2 saved: ESP32-S3 NVS Demo

CMD> 2
=== All data (2 entries) ===
1: Hello World!
2: ESP32-S3 NVS Demo

CMD> 3
Enter entry number to delete:
1
Entry 1 deleted (total 1)

CMD> 2
=== All data (1 entries) ===
1: ESP32-S3 NVS Demo

CMD> 4
All data cleared!

CMD> 2
No data

CMD>
```

7.3 Edge Cases

- **Power Cycle Restart**: Data committed before power loss persists. On boot, the program automatically reads `count` and restores data state.

For example: Operations before power loss

```
CMD> 1
Enter content to save:
Hello World!
Entry 1 saved: Hello World!

CMD> 1
Enter content to save:
ESP32-S3 NVS Demo
Entry 2 saved: ESP32-S3 NVS Demo

CMD> 2
=== All data (2 entries) ===
1: Hello World!
2: ESP32-S3 NVS Demo
```

After power reconnect:

```
================================
   ESP32-S3 NVS Flash Storage
 1 = Append Write
 2 = Read All Data
 3 = Delete Specified Data
 4 = Clear All Data
================================

CMD> 2
=== All data (2 entries) ===
1: Hello World!
2: ESP32-S3 NVS Demo
```

- **Continuous Delete**: After re-indexing, entries are always numbered consecutively from 1 with no index holes.
- **Empty Data Read**: When `count=0`, selecting command 2 outputs `No data`.

---

8. FAQ

Symptom	Cause	Solution
Boot error `ESP_ERR_NVS_NO_FREE_PAGES`	First flash / partition table change / NVS full	Code handles it — auto erase and re-initialize
Data lost after power loss	`nvs_commit()` not called	Ensure `nvs_commit()` after each `nvs_set_xxx()`
String read truncated	`buf_len` too small	Increase buffer, or read actual length first then allocate
`nvs_open` returns failure	Namespace doesn't exist / partition not initialized	Check if `nvs_flash_init()` was called first
NVS partition space insufficient	Too many entries / single value too large	String ≤ 1984 bytes, total capacity depends on partition size (default 24KB)
Terminal interaction no response	UART0 not initialized / serial tool not opened	Confirm terminal connected to correct COM port, baud rate 115200